Bag of Words (BoW)

Here's an example of how the **Bag of Words (BoW)** model works in NLP:

Example:

Suppose we have the following three short documents:

1. **Document 1:** "The cat sat on the mat."
2. **Document 2:** "The dog barked at the cat."
3. **Document 3:** "The cat and the dog played."

Step 1: Tokenization

Breaking down each sentence into words and removing punctuation:

- **Doc 1:** ["The", "cat", "sat", "on", "the", "mat"]
- **Doc 2:** ["The", "dog", "barked", "at", "the", "cat"]
- **Doc 3:** ["The", "cat", "and", "the", "dog", "played"]

Step 2: Vocabulary Creation

The unique words in the entire corpus (after removing duplicates):

["The", "cat", "sat", "on", "the", "mat", "dog", "barked", "at", "and", "played"]

(If we ignore case sensitivity, "The" and "the" are the same.)

Step 3: Counting Frequencies (Term Frequency Vectors)

Creating a table where each row represents a document, and each column represents a word from the vocabulary.

Word	**Doc 1**	**Doc 2**	**Doc 3**
the	2	2	2
cat	1	1	1
sat	1	0	0
on	1	0	0
mat	1	0	0
dog	0	1	1
barked	0	1	0
at	0	1	0
and	0	0	1

Word	Doc 1	Doc 2	Doc 3
played	0	0	1

Step 4: Sparse Representation

Since each document contains only a few words from the vocabulary, many values in the table are **zero**, making the representation sparse.

Step 5: Vector Representation

Each document is now represented as a vector:

- **Doc 1:** [2, 1, 1, 1, 1, 0, 0, 0, 0, 0]
- **Doc 2:** [2, 1, 0, 0, 0, 1, 1, 1, 0, 0]
- **Doc 3:** [2, 1, 0, 0, 0, 1, 0, 0, 1, 1]

These vectors can now be used for **text classification, clustering, and other NLP tasks**.

Would you like me to implement this in Python using **CountVectorizer** from sklearn.feature_extraction.text?

Sparse Matrix Representation in Bag of Words

Since most of the elements in the **Bag of Words (BoW) model** are **zero**, it is memory-inefficient to store them in a dense matrix format. Instead, we use a **sparse matrix representation**, which only stores the non-zero elements and their positions.

Example

Let's use the same three documents:

1. **Document 1:** "The cat sat on the mat."
2. **Document 2:** "The dog barked at the cat."
3. **Document 3:** "The cat and the dog played."

Step 1: Vocabulary & Term Frequency Matrix

Word	Index	Doc 1	Doc 2	Doc 3
the	0	2	2	2
cat	1	1	1	1
sat	2	1	0	0

Word	Index	Doc 1	Doc 2	Doc 3
on	3	1	0	0
mat	4	1	0	0
dog	5	0	1	1
barked	6	0	1	0
at	7	0	1	0
and	8	0	0	1
played	9	0	0	1

Step 2: Sparse Representation (Coordinate Format)

Instead of storing the full matrix, we store only **non-zero values** along with their row (document) and column (word index) positions.

(row, column)	value
(0, 0)	2
(0, 1)	1
(0, 2)	1
(0, 3)	1
(0, 4)	1
(1, 0)	2
(1, 1)	1
(1, 5)	1
(1, 6)	1
(1, 7)	1
(2, 0)	2
(2, 1)	1
(2, 5)	1
(2, 8)	1
(2, 9)	1

Each entry represents (document_index, word_index) = frequency.

Step 3: Implementation in Python

Using **Scipy's sparse matrix**, we can convert the dense matrix into a sparse representation.

```
from sklearn.feature_extraction.text import CountVectorizer
from scipy.sparse import csr_matrix

# Sample documents
docs = [
    "The cat sat on the mat.",
    "The dog barked at the cat.",
    "The cat and the dog played."
]

# Convert text to BoW representation
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(docs)

# Convert to sparse matrix (Compressed Sparse Row format)
sparse_matrix = csr_matrix(X)

# Print sparse representation
print(sparse_matrix)
```

Output:

```
  (0, 1)   1
  (0, 2)   1
  (0, 3)   1
  (0, 4)   1
  (0, 5)   1
  (0, 9)   2
  (1, 0)   1
```

(1, 1) 1

(1, 6) 1

(1, 7) 1

(1, 8) 1

(1, 9) 2

(2, 0) 1

(2, 1) 1

(2, 5) 1

(2, 10) 1

(2, 11) 1

(2, 9) 2

Advantages of Sparse Representation

1. **Memory Efficient** – Stores only non-zero values instead of a full matrix.
2. **Faster Computation** – Saves space and speeds up mathematical operations.
3. **Scalability** – Useful when working with large text datasets.

what does count vectorizer do ?

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(docs)

the **CountVectorizer** from sklearn.feature_extraction.text performs the **Bag of Words (BoW)** transformation. Here's what it does step by step:

Step 1: Tokenization

- It **splits** each document into individual words (tokens).
- It **converts** all words to lowercase (by default).
- It **removes punctuation and special characters**.

For example, given the documents:

docs = [

"The cat sat on the mat.",

"The dog barked at the cat.",

"The cat and the dog played."

]

CountVectorizer() will extract the words:
["the", "cat", "sat", "on", "mat", "dog", "barked", "at", "and", "played"]

Step 2: Vocabulary Creation

- It builds a **vocabulary**, assigning an index to each unique word in the corpus.

Example vocabulary:

{

'and': 0,

'at': 1,

'barked': 2,

'cat': 3,

'dog': 4,

'mat': 5,

'on': 6,

'played': 7,

'sat': 8,

'the': 9

}

(The word order may differ, but this is how words are mapped to indices.)

Step 3: Count Word Frequencies

- It creates a matrix where **each row represents a document**, and **each column represents a word** from the vocabulary.
- The values in the matrix are the **word counts** in each document.

Word	**Doc 1**	**Doc 2**	**Doc 3**
and	0	0	1
at	0	1	0

Word	Doc 1	Doc 2	Doc 3
barked	0	1	0
cat	1	1	1
dog	0	1	1
mat	1	0	0
on	1	0	0
played	0	0	1
sat	1	0	0
the	2	2	2

Step 4: Sparse Representation

- CountVectorizer().fit_transform(docs) returns a **sparse matrix** instead of a full table to save memory.
- You can convert it to a **dense matrix** using:

X.toarray()

Output:

[[0 0 0 1 0 1 1 0 1 2]

[0 1 1 1 1 0 0 0 0 2]

[1 0 0 1 1 0 0 1 0 2]]

Summary of CountVectorizer()

✅ **Tokenizes** words
✅ **Builds vocabulary** of unique words
✅ **Creates a matrix** where rows = documents, columns = words
✅ **Stores word counts** for each document

Would you like to see **TF-IDF (Term Frequency-Inverse Document Frequency)** next? It improves BoW by reducing the importance of common words like "the". 🚀